

Práctica — Data Warehouse en Python y SQLite

Antes de empezar

En esta práctica, crearás una base de datos, definirás la estructura de las tablas necesarias, cargarás los datos proporcionados y realizarás consultas básicas y avanzadas para analizar la información, todo esto usando las tablas del proyecto de la empresa aliada.

Paso a paso:

- Crear una nueva base de datos en SSMS para almacenar todas las tablas de datos proporcionadas.
- Definir la estructura de las tablas necesarias para los conjuntos de datos (DIM_CATEGORY, DIM_PRODUCT, DIM_SEGMENT, DIM_CALENDAR, FACT_SALES).
- Importar los datos en cada tabla utilizando las funciones de importación.
- Realizar consultas básicas para verificar que los datos se han cargado correctamente.
- Ejecutar uniones entre las tablas para combinar la información y obtener insights clave sobre las ventas por categoría, región y periodo de tiempo.
- Guardar y documentar el proceso, incluyendo capturas de pantalla de las consultas y los resultados obtenidos.

1. Crear la base de datos

```
import sqlite3
import pandas as pd
import os

# Crea el archivo Proyecto_Ventas.db si no existe
os.chdir("C:/Users/LUIS HERNANDEZ/Desktop/EBAC")
conn = sqlite3.connect("C:/Users/LUIS HERNANDEZ/Desktop/EBAC/Proyecto_Ventas.db")
cur = conn.cursor()
print("Base de datos Proyecto_Ventas.db lista.")
```

Base de datos Proyecto_Ventas.db lista.

2. Definir la estructura de las tablas

```
# Se elimina cada tabla si ya existía, para poder re-ejecutar el notebook
for t in ["FACT_SALES", "DIM_PRODUCT", "DIM_SEGMENT", "DIM_CALENDAR", "DIM_CATEGORY"]:
    cur.execute(f"DROP TABLE IF EXISTS {t};")

cur.execute("""
CREATE TABLE DIM_CATEGORY (
    ID_CATEGORY INTEGER PRIMARY KEY,
    CATEGORY    TEXT
);""")

cur.execute("""
CREATE TABLE DIM_CALENDAR (
    WEEK        TEXT PRIMARY KEY,
    YEAR        INTEGER,
    MONTH       INTEGER,
    WEEK_NUMBER INTEGER,
    DATE        TEXT
);""")

cur.execute("""
CREATE TABLE DIM_PRODUCT (
    MANUFACTURER TEXT,
    BRAND        TEXT,
    ITEM         TEXT,
```

```

        ITEM_DESCRIPTION TEXT,
        CATEGORY         INTEGER,
        FORMAT            TEXT,
        ATTR1             TEXT,
        ATTR2             TEXT,
        ATTR3             TEXT
    );"""

cur.execute("""
CREATE TABLE DIM_SEGMENT (
    CATEGORY INTEGER,
    ATTR1    TEXT,
    ATTR2    TEXT,
    ATTR3    TEXT,
    FORMAT   TEXT,
    SEGMENT  TEXT
);""")

cur.execute("""
CREATE TABLE FACT_SALES (
    WEEK                TEXT,
    ITEM_CODE           TEXT,
    TOTAL_UNIT_SALES    REAL,
    TOTAL_VALUE_SALES   REAL,
    TOTAL_UNIT_AVG_WEEKLY_SALES REAL,
    REGION              TEXT
);""")

conn.commit()
print("5 tablas creadas.")

```

5 tablas creadas.

3. Importar los datos

Se leen los archivos con pandas (que maneja tanto .csv como .xlsx) y se cargan en su tabla con `to_sql(..., if_exists="append")`. A la categoría se le quitan los espacios sobrantes del texto.

```

# Leer cada archivo con la función adecuada
dim_category = pd.read_csv("DIM_CATEGORY.csv")
dim_segment  = pd.read_excel("DIM_SEGMENT.xlsx")
dim_calendar = pd.read_excel("DIM_CALENDAR (4).xlsx")
dim_product  = pd.read_excel("DIM_PRODUCT.xlsx")
fact_sales   = pd.read_csv("FACT_SALES.csv")

# Limpieza: quitar espacios/saltos de línea sobrantes en el nombre de categoría
dim_category["CATEGORY"] = dim_category["CATEGORY"].str.strip()

# Insertar en cada tabla
dim_category.to_sql("DIM_CATEGORY", conn, if_exists="append", index=False)
dim_calendar.to_sql("DIM_CALENDAR", conn, if_exists="append", index=False)
dim_product.to_sql("DIM_PRODUCT", conn, if_exists="append", index=False)
dim_segment.to_sql("DIM_SEGMENT", conn, if_exists="append", index=False)
fact_sales.to_sql("FACT_SALES", conn, if_exists="append", index=False)
conn.commit()
print("Datos importados en las 5 tablas.")

```

Datos importados en las 5 tablas.

4. Consultas básicas de verificación

```

# Conteo de filas por tabla

```

```
consulta = """
SELECT 'DIM_CATEGORY' AS Tabla, COUNT(*) AS Filas FROM DIM_CATEGORY
UNION ALL SELECT 'DIM_CALENDAR', COUNT(*) FROM DIM_CALENDAR
UNION ALL SELECT 'DIM_PRODUCT', COUNT(*) FROM DIM_PRODUCT
UNION ALL SELECT 'DIM_SEGMENT', COUNT(*) FROM DIM_SEGMENT
UNION ALL SELECT 'FACT_SALES', COUNT(*) FROM FACT_SALES;
"""
pd.read_sql(consulta, conn)
```

	Tabla	Filas
0	DIM_CATEGORY	5
1	DIM_CALENDAR	156
2	DIM_PRODUCT	505
3	DIM_SEGMENT	53
4	FACT_SALES	122002

```
# Vistazo a las primeras filas de la tabla de hechos
pd.read_sql("SELECT * FROM FACT_SALES LIMIT 5;", conn)
```

	WEEK	ITEM_CODE	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES	TOTAL_UNIT_AVG_WEEKLY_SALES	REGION
0	34-22	7501058792808BP2	0.006	0.139	1.000	TOTAL AUTOS AREA 5
1	34-22	7501058715883	0.487	116.519	2.916	TOTAL AUTOS AREA 5
2	34-22	7702626213774	1.391	68.453	5.171	TOTAL AUTOS AREA 5
3	34-22	7501058716422	0.022	1.481	1.833	TOTAL AUTOS AREA 5
4	34-22	7501058784353	2.037	182.839	5.375	TOTAL AUTOS AREA 5

```
# Vistazo a los productos
pd.read_sql("SELECT * FROM DIM_PRODUCT LIMIT 5;", conn)
```

	MANUFACTURER	BRAND	ITEM	ITEM_DESCRIPTION	CATEGORY	FORMAT	ATTR1	ATTR2	ATTR3
0	INDS. ALEN	CLORALEX	0000075000592	CLORALEX EL RENDIDOR BOT.PLAST. 250ML NAL. 000...	1	LIQUIDO	COLORO	COLORO	1 DEFINI...
1	INDS. ALEN	CLORALEX	0000075000608	CLORALEX EL RENDIDOR BOT.PLAST. 500ML NAL. 000...	1	LIQUIDO	COLORO	COLORO	1 DEFINI...
2	INDS. ALEN	CLORALEX	0000075000615	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL. 000...	1	LIQUIDO	COLORO	COLORO	1 DEFINI...
3	INDS. ALEN	CLORALEX	0000075000622	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL. 000...	1	LIQUIDO	COLORO	COLORO	1 DEFINI...

4	INDS. ALEN	CLORALEX	0000075000639	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000...	1	LIQUIDO	COLORO	COLORO	1 DEFINI
---	------------	----------	---------------	--	---	---------	--------	--------	-------------

5. Uniones (JOINS) para obtener insights

5.1 Ventas por categoría

```
pd.read_sql("""
    SELECT c.CATEGORY AS Categoria,
           ROUND(SUM(f.TOTAL_VALUE_SALES), 2) AS Valor_Total,
           ROUND(SUM(f.TOTAL_UNIT_SALES), 2) AS Unidades_Total
    FROM FACT_SALES f
    JOIN DIM_PRODUCT p ON f.ITEM_CODE = p.ITEM
    JOIN DIM_CATEGORY c ON p.CATEGORY = c.ID_CATEGORY
    GROUP BY c.CATEGORY
    ORDER BY Valor_Total DESC;
""", conn)
```

	Categoria	Valor_Total	Unidades_Total
0	FABRIC TREATMENT and SANIT	11042859.89	391760.23

5.2 Ventas por región

```
pd.read_sql("""
    SELECT REGION,
           ROUND(SUM(TOTAL_VALUE_SALES), 2) AS Valor_Total,
           ROUND(SUM(TOTAL_UNIT_SALES), 2) AS Unidades_Total
    FROM FACT_SALES
    GROUP BY REGION
    ORDER BY Valor_Total DESC;
""", conn)
```

	REGION	Valor_Total	Unidades_Total
0	TOTAL AUTOS SCANNING MEXICO	5521429.32	195880.05
1	TOTAL AUTOS AREA 2	1188796.15	45319.92
2	TOTAL AUTOS AREA 5	1153335.54	36266.90
3	TOTAL AUTOS AREA 6	983957.57	38585.22
4	TOTAL AUTOS AREA 3	803655.34	25896.05
5	TOTAL AUTOS AREA 1	714249.98	25791.93
6	TOTAL AUTOS AREA 4	677436.00	24020.15

5.3 Ventas por periodo (año y mes)

```
pd.read_sql("""
    SELECT cal.YEAR AS Anio, cal.MONTH AS Mes,
           ROUND(SUM(f.TOTAL_VALUE_SALES), 2) AS Valor_Total
    FROM FACT_SALES f
    JOIN DIM_CALENDAR cal ON f.WEEK = cal.WEEK
    GROUP BY cal.YEAR, cal.MONTH
    ORDER BY cal.YEAR, cal.MONTH;
""", conn)
```

	Anio	Mes	Valor_Total
0	2022	1	678440.71
1	2022	2	494535.23
2	2022	3	510052.35
3	2022	4	522004.23
4	2022	5	674949.56
5	2022	6	603800.02
6	2022	7	702062.39
7	2022	8	545441.05
8	2022	9	561226.13
9	2022	10	668951.12
10	2022	11	526050.10
11	2022	12	521106.15
12	2023	1	584020.63
13	2023	2	523571.53
14	2023	3	541149.85
15	2023	4	546329.64
16	2023	5	711883.29
17	2023	6	654159.28
18	2023	7	473126.63

5.4 Top 10 marcas por valor de ventas

```
pd.read_sql("""
    SELECT p.BRAND AS Marca,
           ROUND(SUM(f.TOTAL_VALUE_SALES), 2) AS Valor_Total
    FROM FACT_SALES f
    JOIN DIM_PRODUCT p ON f.ITEM_CODE = p.ITEM
    GROUP BY p.BRAND
    ORDER BY Valor_Total DESC
    LIMIT 10;
""", conn)
```

	Marca	Valor_Total
0	CLORALEX	5393618.51
1	VANISH	2168289.52
2	CLOROX	1371751.72
3	BLANCATEL	652484.02
4	OXI CLEAN	436970.72
5	DR.BECKMANN	279719.23
6	BANDERA	269936.90
7	LOS PATITOS	142745.66
8	CLARASOL	141012.17
9	LYSOL	70557.35

5.5 Ventas por segmento (usando la tabla puente DIM_SEGMENT)

```
pd.read_sql("""
    SELECT s.SEGMENT AS Segmento,
           ROUND(SUM(f.TOTAL_VALUE_SALES), 2) AS Valor_Total
    FROM FACT_SALES f
    JOIN DIM_SEGMENT s ON f.ITEM_CODE = s.ITEM
    GROUP BY s.SEGMENT
    ORDER BY Valor_Total DESC
    LIMIT 10;
""", conn)
```

```

FROM FACT_SALES f
JOIN DIM_PRODUCT p ON f.ITEM_CODE = p.ITEM
JOIN DIM_SEGMENT s
    ON p.CATEGORY = s.CATEGORY AND p.ATTR1 = s.ATTR1
    AND p.ATTR2 = s.ATTR2 AND p.ATTR3 = s.ATTR3 AND p.FORMAT = s.FORMAT
GROUP BY s.SEGMENT
ORDER BY Valor_Total DESC;
""" , conn)

```

	Segmento	Valor_Total
0	BLEACH	7581283.77
1	LIQUID & GEL	1454056.07
2	POWDER	1035021.68
3	PRETREAT	632002.03
4	BAR	243539.23
5	SANITIZER	81915.70
6	OTHERS	15041.42

```

# Cerrar la conexión
conn.close()

```

6. Conclusiones

- La base Proyecto_Ventas.db se creó con las 5 tablas del esquema estrella y se cargaron correctamente (CATEGORY = 5, CALENDAR = 156, PRODUCT = 505, SEGMENT = 53, FACT_SALES = 122,002 filas).
- **Por categoría:** las ventas del conjunto de hechos corresponden a la categoría *FABRIC TREATMENT and SANIT*.
- **Por región:** *TOTAL AUTOS SCANNING MEXICO* concentra el mayor valor (es el total nacional, suma de las 6 áreas).
- **Por periodo:** 2022 fue el año con mayor valor de ventas, seguido de 2023.
- **Por marca:** *CLORALEX*, *VANISH* y *CLOROX* lideran el valor de ventas.